SNAIL Shell

Group 16 | Custom Unix-like Shell in C

A Unix-like shell focused on safe file operations, modular POSIX design, and a configurable user experience.

Presentation Focus

- REPL + parser + executor
- Safe file dump
- Interactive removal
- Editable shell UI

Built from the Snail codebase, progression tracker workbook, SRS history, testing sheet, and the assignment presentation rubric.

Objective and Motivation

Objective

- Build a custom Unix-like shell in C with core shell behavior such as a REPL, command parsing, built-ins, external execution, redirection, and pipelines.
- Keep the design modular so parsing, execution, prompt rendering, dump handling, and UI logic evolve independently.
- Make the shell safer and easier to use than a minimal classroom shell.

Motivation

- Learn low-level OS concepts through fork, execvp, waitpid, dup2, signals, and raw terminal input.
- Reduce accidental file loss by routing deletion through a dump instead of immediate permanent removal.
- Give users a shell they can personalize through an editable config and a lightweight UI launcher.

Built from the Snail codebase, progression tracker workbook, SRS history, testing sheet, and the assignment presentation rubric.

Challenges and How We Solved Them

Key Challenges

- Coordinating parser, executor, built-ins, redirection, and pipelines without tightly coupling modules.
- Handling raw terminal input, history navigation, prompt redraws, and Ctrl+C safely inside an interactive REPL.
- Designing deletion behavior that stays safe while still feeling like normal shell usage.
- Keeping the shell customizable across both terminal mode and the separate UI launcher.

Our Solutions

- Used a modular C structure with separate source files for parsing, execution, prompt, config, rm, mv, dump, and UI.
- Added raw-mode helpers, prompt refresh logic, signal handling, command history, and redraw support in the REPL.
- Implemented a Snail dump folder with metadata and 48-hour cleanup instead of destructive delete-by-default.
- Loaded shared settings from snail.conf / ~/snailShellrc so visual behavior can be edited without recompiling.

Built from the Snail codebase, progression tracker workbook, SRS history, testing sheet, and the assignment presentation rubric.

Novelty in Our Work

What Makes Snail Different

- File dump instead of immediate delete: rm routes files into ~/.snailDump with metadata, giving users a safety net.
- Interactive removal mode: rm -i ranks files, lets users select with the keyboard, and confirms deletion with reclaimable size feedback.
- Editable shell UI: a Tkinter-based launcher starts the shell in a configurable window whose theme and font come from the same config flow.
- Configurable prompt: path style, separator, git branch, exit status, duration, time, and color theme are user-editable.

Why This Matters

- Traditional student shells often stop at command execution; Snail adds safety and usability as first-class design goals.
- The novelty is not one feature but the combination of safe deletion, guided interaction, and personalization in a low-level shell project.
- These choices make the project more deployable, demo-friendly, and closer to real user workflows.

Built from the Snail codebase, progression tracker workbook, SRS history, testing sheet, and the assignment presentation rubric.

Demo Flow

Suggested Live Demo

- 1. Launch Snail and show the themed prompt with git branch, time, duration, and path formatting.
- 2. Run built-ins like cd, ls, mkdir, touch, tree, and reloadConfig to show modular shell behavior.
- 3. Show an external command plus redirection and a simple pipeline to prove POSIX execution flow.
- 4. Run rm on a sample file to explain the dump-based safety mechanism instead of permanent loss.
- 5. Run rm -i on a directory to show ranked selection, keyboard navigation, and confirmation flow.
- 6. Open the Python UI launcher to demonstrate the editable shell UI and shared theme settings.

Code Anchors

- Prompt and metadata: src/prompt.c
- Interactive REPL and history: src/repl.c
- Built-in dispatch and pipes: src/executor.c
- Safe dump workflow: src/fileDump.c + src/rmCommand.c
- Editable shell UI: ui_snail/intro.py

Built from the Snail codebase, progression tracker workbook, SRS history, testing sheet, and the assignment presentation rubric.

Analysis and Validation

Validation backed by tracker data, testing logs, and implementation evidence.

13
manual test cases listed in the testing sheet

15
autonomous regression checks tracked across versions

48h
retention window for dump cleanup

9
planned sprint themes recorded in the tracker

Analysis

- The testing workbook records PASS results for built-ins, parser behavior, error handling, external execution, long-input safety, and memory checks.
- Autonomous checks show maturity from earlier versions to v3, especially around dump routing, recursive cleanup, config loading, prompt behavior, and mv rollback.
- The tracker also shows a clean progression from shell core to parser, built-ins, external execution, redirection, dump infrastructure, and UI support.
- Overall, the project demonstrates both functional depth and process discipline through SRS updates, sprint

Future Scope

Next Technical Steps

- Expose dump restore as a user-facing command with listing and search support.
- Add safer directory-aware interactive cleanup and optional dry-run mode.
- Improve portability by abstracting Linux-specific calls and preparing a clearer Windows-compatible path.
- Extend the UI with tabs, better log panes, and richer settings editing.

Quality and Product Growth

- Add autocomplete, aliases, and smarter help/documentation inside the shell.
- Strengthen crash recovery and terminal restoration using broader signal handling.
- Expand automated testing and benchmark scripts around pipelines, redirection, and dump edge cases.
- Consider compression or smarter retention policies for large dump contents.

Built from the Snail codebase, progression tracker workbook, SRS history, testing sheet, and the assignment presentation rubric.

Individual Contribution

Mapped from the implementation sheet, sprint ownership, and SRS/testing records.

Aditya

- Shell core and input handling in the tracker, plus executor integration and external command execution.
- Major ownership around redirection, dump infrastructure, and several integration tasks across sprints.

Ishaan

- Core built-ins such as ls, cd, rm, mv, plus implementation of commandHistory and the interactive removal workflow in rm -i.
- Handled major data flow for pipelines and contributed to the safety-focused shell behavior through dump-related evolution.

Koushik

- Implemented signal handling and stability-focused behavior, and also contributed touch, mkdir, and rmdir in the command set.
- Had stronger focus on software testing, validation alignment, and sprint/process oversight reflected in the tracker and testing sheets.

Arpit

- Worked on shell core support, touch/tree-related implementation, UI involvement, and direct file-directory side features used in the shell workflow.
- Also contributed strongly to SRS, UML, testing documentation, and helped connect implementation output with presentation-ready validation artifacts.

Built from the Snail codebase, progression tracker workbook, SRS history, testing sheet, and the assignment presentation rubric.